

Kubernetes (K8s) Cluster Architecture

KUBERNETES CLUSTER ARCHITECTURE AND CONTAINERIZATION STRATEGY

Project: Monolith to Microservices Transition Author: David Lin, Principal DevOps Engineer Date: August 18, 2026

- 1. Overview** This document defines the baseline architecture for migrating the legacy Java monolithic application into containerized microservices orchestrated via Amazon Elastic Kubernetes Service (EKS). The transition will improve deployment velocity, resource utilization, and horizontal scalability.
- 2. Cluster Topology** We will provision three distinct EKS clusters: Development, Staging, and Production. The Production cluster will span three Availability Zones (AZs) in `us-east-1`.
 - Control Plane: Managed by AWS (EKS). We will enforce API server endpoint access strictly via private VPC subnets to mitigate external attack vectors.
 - Worker Nodes: A mix of On-Demand and Spot instances managed via Karpenter. Spot instances will be aggressively utilized for stateless, fault-tolerant workloads (e.g., image rendering pods), while On-Demand instances will host stateful services and the ingress controllers.
- 3. Networking and Ingress** Pod networking will be handled by the AWS VPC CNI plugin, providing native IP addresses to each container. For external traffic routing, we will deploy the NGINX Ingress Controller. Internal service-to-service communication will be secured using an Istio Service Mesh, which enforces mutual TLS (mTLS) automatically without requiring application code changes.
- 4. CI/CD Integration** Deployment will be entirely GitOps-driven using ArgoCD. Developers will merge code into the `main` branch, triggering a GitHub Actions pipeline to build the Docker image and push it to the Elastic Container Registry (ECR). The pipeline will then update the Helm chart manifests. ArgoCD, constantly monitoring the repository, will detect the drift and automatically reconcile the cluster state by rolling out the new pods using a progressive Canary deployment strategy.
- 5. Observability** Prometheus and Grafana will be deployed within the cluster to scrape and visualize metrics. Fluent Bit will be configured as a DaemonSet to collect pod logs and forward them to our centralized Elasticsearch instance.